

Caracterizando a atividade de code review no GitHub

Pedro Negri Leão Lambert

07/05/2026

Link para repo/dados:

<https://github.com/Pedro-nll/LabExperimentacaoDeSoftware/tree/enunciado3>

Qualidade de repositórios Java populares.....	1
1 - Introdução.....	2
1.1 - Contextualização.....	2
1.1.1 - Restrições e riscos.....	2
1.2 - Problema foco do experimento.....	2
1.3 - Questões de Pesquisa.....	2
1.4 - Hipóteses.....	2
2 - Objetivo.....	2
2.1 - Objetivo principal.....	2
2.2 - Objetivos específicos.....	2
3 - Metodologia.....	2
3.1 - Passo a passo do experimento.....	2
3.2 - Decisões.....	2
3.3 - Materiais utilizados.....	2
3.4 - Métodos utilizados.....	2
3.5 - Métricas e suas unidades.....	2
4 - Visualização dos Resultados.....	2
5 - Discussão dos Resultados.....	2
6 - Conclusão.....	2
6.1 - Trabalhos futuros.....	2
7 - Reprodutibilidade.....	3

1 - Introdução

1.1 - Contextualização

A prática de *code review* é uma etapa importante no processo de desenvolvimento de software, especialmente em projetos colaborativos e *open-source*. Em repositórios hospedados no GitHub, essa prática ocorre principalmente por meio de *Pull Requests* (PRs), nos quais uma contribuição é submetida, discutida, revisada e, posteriormente, integrada ou rejeitada pelos mantenedores do projeto.

Em projetos populares, esse processo tende a envolver diferentes perfis de contribuidores, revisores humanos, ferramentas automatizadas de verificação e múltiplas interações antes da decisão final. Dessa forma, o estudo da atividade de *code review* permite compreender como características dos PRs se relacionam com o resultado final da revisão e com a intensidade do processo de análise.

Neste laboratório, o foco está na caracterização da atividade de *code review* em repositórios populares do GitHub. O estudo considera PRs como unidade de análise e busca observar se características como tamanho, tempo de análise, descrição e interações apresentam relação com duas dimensões principais: o feedback final recebido pelo PR e o número de revisões realizadas.

1.1.1 - Restrições e riscos

Apesar de o planejamento inicial considerar os 200 repositórios mais populares do GitHub que atendessem aos critérios definidos, a análise foi realizada sobre o subconjunto efetivamente coletado até o momento da geração do relatório. O dataset analisado contém **6.768 PRs provenientes de 132 repositórios**, o que caracteriza uma amostra parcial, porém suficiente para uma análise exploratória inicial.

A primeira limitação diz respeito à classificação do feedback final. Embora o processo busque normalizar os resultados das revisões em categorias comparáveis, **o comportamento real de um PR pode envolver múltiplas revisões, comentários, aprovações, rejeições ou alterações posteriores**. Assim, a categoria final utilizada na análise representa uma simplificação do processo de revisão.

Também é importante destacar que as **métricas analisadas não explicam causalidade**. Relações estatísticas entre tamanho, tempo, descrição, interações, feedback e número de revisões devem ser interpretadas como associações observadas na **amostra**, e não como prova de que uma **variável causa diretamente a outra**.

Por fim, algumas métricas de PRs tendem a apresentar distribuições assimétricas, com poucos PRs muito grandes ou muito demorados. Por esse motivo, a análise dos resultados deve considerar a **presença de outliers**, e a interpretação das correlações deve priorizar medidas adequadas para dados assimétricos, como a correlação de Spearman, sem descartar a comparação com Pearson quando útil.

1.2 - Problema foco do experimento

O problema foco deste experimento é compreender como determinadas **características de Pull Requests se relacionam com a atividade de code review** em repositórios populares do GitHub.

Em especial, busca-se investigar se PRs maiores, mais demorados, mais descritivos ou com maior número de interações apresentam diferenças quanto ao feedback final recebido e quanto ao número de revisões realizadas. Essa análise permite observar padrões do processo de revisão em projetos open-source populares e discutir quais fatores podem estar associados a revisões mais intensas, demoradas ou com resultados distintos.

Assim, o experimento **não busca avaliar a qualidade do código em si, mas sim caracterizar aspectos do processo de revisão dos PRs**, considerando métricas extraídas diretamente da atividade registrada no GitHub.

1.3 - Questões de Pesquisa

As questões de pesquisa foram organizadas em duas dimensões: a primeira relacionada ao feedback final das revisões e a segunda relacionada ao número de revisões realizadas.

Dimensão	RQ	Questão de Pesquisa
Feedback final das revisões	RQ01	Qual a relação entre o tamanho dos PRs e o feedback final das revisões?
Feedback final das revisões	RQ02	Qual a relação entre o tempo de análise dos PRs e o feedback final das revisões?
Feedback final das revisões	RQ03	Qual a relação entre a descrição dos PRs e o feedback final das revisões?
Feedback final das revisões	RQ04	Qual a relação entre as interações nos PRs e o feedback final das revisões?
Número de revisões	RQ05	Qual a relação entre o tamanho dos PRs e o número de revisões realizadas?
Número de revisões	RQ06	Qual a relação entre o tempo de análise dos PRs e o número de revisões realizadas?
Número de revisões	RQ07	Qual a relação entre a descrição dos PRs e o número de revisões realizadas?
Número de revisões	RQ08	Qual a relação entre as interações nos PRs e o número de revisões realizadas?

1.4 - Hipóteses

Antes da execução completa da análise, foram definidas hipóteses informais para orientar a interpretação dos resultados. Essas hipóteses representam expectativas iniciais sobre o comportamento dos dados e serão posteriormente comparadas com os valores obtidos.

RQ	Hipótese Inicial
RQ0 1	O tamanho do PR não deve influenciar fortemente o feedback final, pois PRs grandes e pequenos podem ser aprovados ou rejeitados dependendo do contexto da alteração.
RQ0 2	PRs com maior tempo de análise podem indicar maior complexidade ou dificuldades no processo de revisão, podendo ter relação negativa com feedbacks positivos.
RQ0 3	O tamanho da descrição não deve influenciar diretamente o feedback final, já que uma descrição maior não necessariamente indica maior qualidade da contribuição.
RQ0 4	PRs com mais interações podem envolver temas mais sensíveis ou controversos, podendo apresentar maior chance de feedbacks neutros ou negativos.
RQ0 5	PRs maiores podem demandar mais esforço de análise, mas também podem desestimular a participação de muitos revisores.
RQ0 6	PRs que permanecem mais tempo em análise podem atrair menos participação direta ou refletir processos de revisão mais lentos.
RQ0 7	PRs com descrições mais completas podem facilitar o entendimento da contribuição e atrair maior participação no processo de revisão.
RQ0 8	PRs com mais interações devem apresentar maior número de revisões, pois comentários, participantes e revisões fazem parte de um processo de discussão mais ativo.

2 - Objetivo

2.1 - Objetivo principal

O objetivo principal deste experimento é analisar a atividade de *code review* em Pull Requests de repositórios populares do GitHub, buscando **compreender como características dos PRs se relacionam com o feedback final das revisões e com o número de revisões realizadas.**

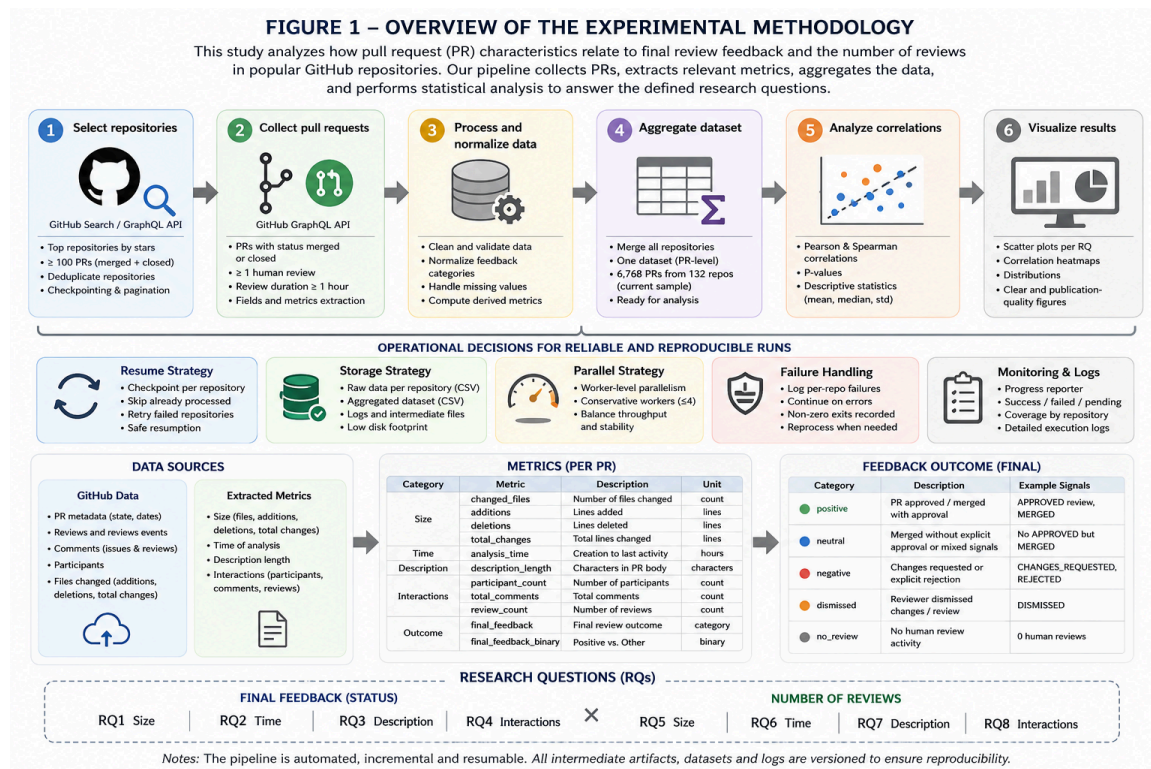
A análise considera como **unidade principal cada Pull Request coletado**, observando métricas relacionadas ao tamanho da alteração, ao tempo de análise, à descrição do PR e às interações ocorridas durante o processo de revisão. Dessa forma, busca-se caracterizar padrões da atividade de revisão de código em projetos *open-source* populares e avaliar se determinados atributos dos PRs estão associados a revisões mais intensas, demoradas ou com diferentes resultados finais.

2.2 - Objetivos específicos

Objetivo	Descrição
OE01	Coletar dados de Pull Requests submetidos a repositórios populares do GitHub, considerando projetos com volume relevante de PRs e presença de revisões humanas .
OE02	Extraí métricas relacionadas ao tamanho dos PRs , incluindo número de arquivos alterados, linhas adicionadas, linhas removidas e total de alterações.
OE03	Calcular o tempo de análise de cada PR , considerando o intervalo entre sua criação e sua última atividade registrada.
OE04	Medir o tamanho da descrição dos PRs a partir da quantidade de caracteres presentes no corpo da descrição em Markdown.
OE05	Quantificar as interações ocorridas nos PRs , considerando participantes, comentários e revisões associadas ao processo de análise.
OE06	Classificar o feedback final das revisões em categorias comparáveis, permitindo relacionar o resultado da revisão com as demais métricas coletadas.
OE07	Aplicar estatísticas descritivas e testes de correlação para investigar as relações propostas nas questões de pesquisa.
OE08	Interpretar os resultados obtidos à luz das hipóteses iniciais , discutindo possíveis associações entre características dos PRs e comportamento do processo de code review.

3 - Metodologia

A metodologia deste experimento foi organizada como um pipeline automatizado de coleta, consolidação e análise de Pull Requests de repositórios populares do GitHub. O estudo utiliza PRs como unidade de análise e relaciona suas características principais com o feedback final das revisões e com o número de revisões realizadas. O enunciado define que o dataset deve considerar PRs de repositórios populares, com pelo menos 100 PRs e com atividade de revisão, além de exigir a análise de correlação com teste estatístico adequado.



A Figura 1 apresenta uma visão geral da metodologia experimental. O fluxo começa com a seleção dos repositórios, passa pela coleta dos Pull Requests, normalização dos dados, agregação em um dataset único, análise estatística das correlações e geração dos gráficos finais. A figura também resume as principais decisões operacionais do pipeline, como estratégia de retomada, armazenamento dos arquivos, paralelismo, tratamento de falhas e monitoramento da execução. Por fim, ela apresenta as fontes de dados, as métricas utilizadas por PR e a separação das questões de pesquisa entre feedback final e número de revisões.

3.1 - Passo a passo do experimento

O experimento foi executado a partir de um fluxo incremental e reproduzível. Primeiro, foram selecionados repositórios populares do GitHub que atendiam aos critérios definidos para o estudo. Em seguida, para cada repositório, foram coletados Pull Requests com seus respectivos metadados, informações de revisão, comentários, participantes e métricas de tamanho.

Após a coleta, os arquivos individuais de cada repositório foram consolidados em um único dataset agregado. Esse dataset foi então normalizado, removendo ou tratando valores inválidos e preparando as variáveis necessárias para a análise estatística. Por fim, foram geradas estatísticas descritivas, tabelas de correlação e gráficos exploratórios para responder às questões de pesquisa. De forma resumida:

Etapas	Descrição
1	Seleção dos repositórios populares do GitHub.
2	Coleta dos Pull Requests e dados de revisão.
3	Extração das métricas de tamanho, tempo, descrição...
4	Consolidação dos arquivos por repositório em um dataset único.
5	Limpeza e normalização dos dados coletados.
6	Cálculo de estatísticas descritivas e correlações.
7	Geração dos gráficos utilizados na análise dos resultados.

3.2 - Decisões

Algumas decisões foram tomadas para tornar a execução do experimento mais estável e adequada para uma coleta longa. A primeira delas foi tratar o **Pull Request como unidade principal de análise, em vez do repositório**. Assim, **cada linha do dataset representa um PR**, enquanto os repositórios servem como origem dos dados e como forma de validar a cobertura da coleta.

Também foi adotada uma estratégia de **execução com checkpoints**, permitindo que repositórios já processados fossem ignorados em execuções futuras. Repositórios com falha permaneceram disponíveis para novas tentativas, reduzindo a perda de dados causada por erros temporários de rede, limites da API ou interrupções locais. O pipeline também foi projetado para consolidar os dados ao final da coleta, gerando um arquivo único para análise estatística.

Outra decisão relevante foi **utilizar tanto Pearson quanto Spearman** nas análises de correlação. **Pearson foi mantido como medida de associação linear**, enquanto **Spearman foi utilizado por ser mais adequado para distribuições assimétricas e presença de outliers**, comportamento comum em métricas de PRs, como número de linhas alteradas, tempo de análise e comentários.

3.3 - Materiais utilizados

Os principais materiais utilizados no experimento foram os **dados públicos de Pull Requests disponíveis no GitHub e os scripts desenvolvidos para coleta**, agregação, análise e visualização dos dados. A coleta foi realizada por meio da API do GitHub/GraphQL, extraindo informações dos PRs, revisões, comentários, participantes e metadados dos repositórios.

Os artefatos produzidos pelo pipeline incluem o dataset agregado de PRs, arquivos de estatísticas descritivas, distribuição de feedbacks, tabelas de correlação e gráficos finais. Segundo a documentação da metodologia, os principais arquivos de saída incluem `aggregated_prs.csv`, `descriptive_statistics.csv`, `feedback_distribution.csv`, os arquivos de correlação para as RQs e os gráficos gerados para análise e publicação.

3.4 - Métodos utilizados

O método utilizado é exploratório e quantitativo. A análise parte da coleta de dados reais de Pull Requests e busca identificar associações entre variáveis por meio de estatísticas descritivas e testes de correlação. Como o objetivo é caracterizar padrões da atividade de code review, os resultados são interpretados como relações observadas na amostra, e não como relações causais.

Para cada métrica, foram calculados valores de tendência central e dispersão, como média, mediana e desvio padrão. Em seguida, foram aplicadas correlações de **Pearson e Spearman** para avaliar a relação entre as variáveis independentes, como tamanho, tempo, descrição e interações, e as variáveis de interesse, como feedback final e número de revisões.

O uso de Spearman é especialmente relevante porque as métricas de Pull Requests tendem a ser assimétricas: poucos PRs concentram valores muito altos de linhas alteradas, comentários ou duração. Dessa forma, a análise por postos reduz o impacto de outliers e complementa a interpretação fornecida por Pearson.

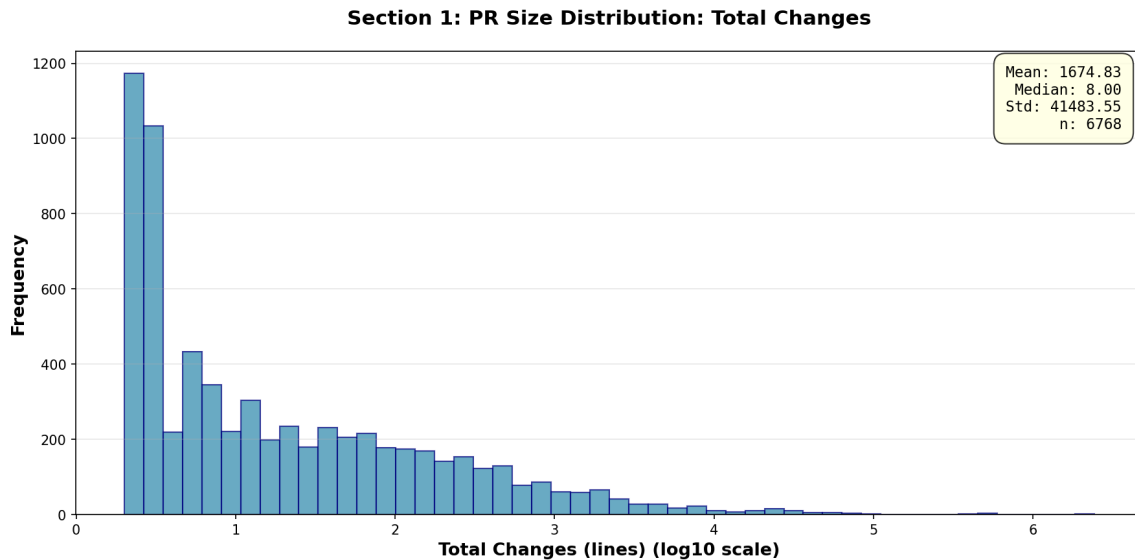
3.5 - Métricas e suas unidades

As métricas utilizadas foram definidas a partir das dimensões indicadas no enunciado: tamanho, tempo de análise, descrição e interações. Além delas, foram consideradas as variáveis relacionadas ao feedback final e ao número de revisões, que representam os principais resultados analisados nas questões de pesquisa.

Categoria	Métrica	Descrição	Unidade
Tamanho	<code>changed_files</code>	Número de arquivos alterados no PR.	Contagem
Tamanho	<code>additions</code>	Total de linhas adicionadas.	Linhas
Tamanho	<code>deletions</code>	Total de linhas removidas.	Linhas
Tamanho	<code>total_changes</code>	Soma de linhas adicionadas e removidas.	Linhas
Tempo	<code>analysis_time_hours</code>	Intervalo entre a criação do PR e a última atividade registrada.	Horas
Descrição	<code>description_length</code>	Tamanho do corpo da descrição do PR em Markdown.	Caracteres
Interações	<code>total_comments</code>	Total de comentários associados ao PR.	Contagem
Interações	<code>participant_count</code>	Número de participantes envolvidos no PR.	Contagem
Revisões	<code>human_review_count / review_count</code>	Número de revisões humanas registradas no PR.	Contagem
Feedback	<code>final_feedback</code>	Classificação final do resultado da revisão.	Categoria

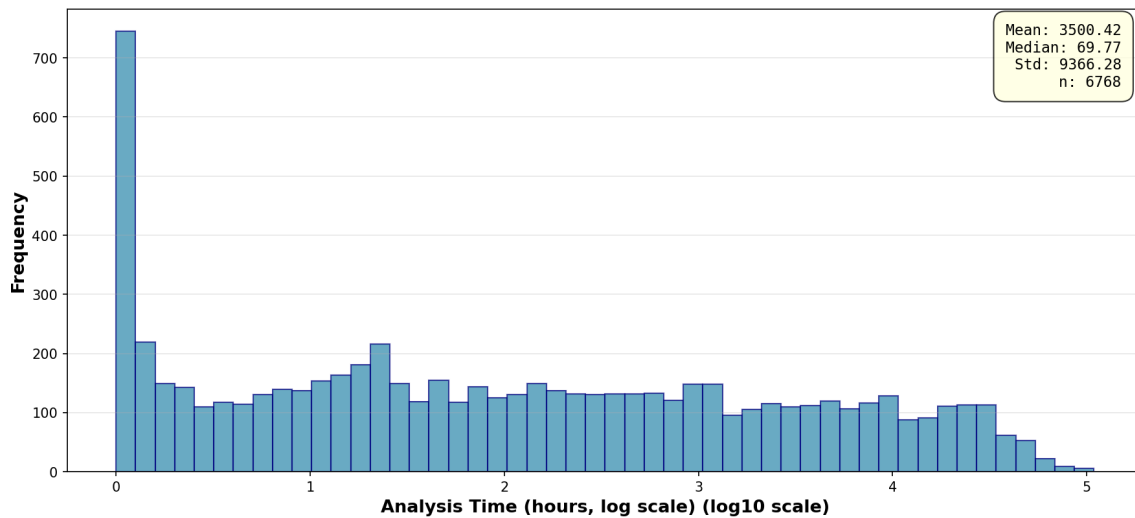
4 - Visualização dos Resultados

Esta seção apresenta as **principais visualizações** geradas a partir do dataset analisado. As figuras foram organizadas para **primeiro caracterizar a amostra geral** e, em seguida, **apoiar a interpretação das questões de pesquisa** relacionadas ao feedback final e ao número de revisões. **O resumo estatístico indica que a amostra possui forte assimetria em várias métricas**, como tamanho dos PRs, tempo de análise e interações, o que justifica o uso de visualizações e correlações robustas para interpretar os resultados.



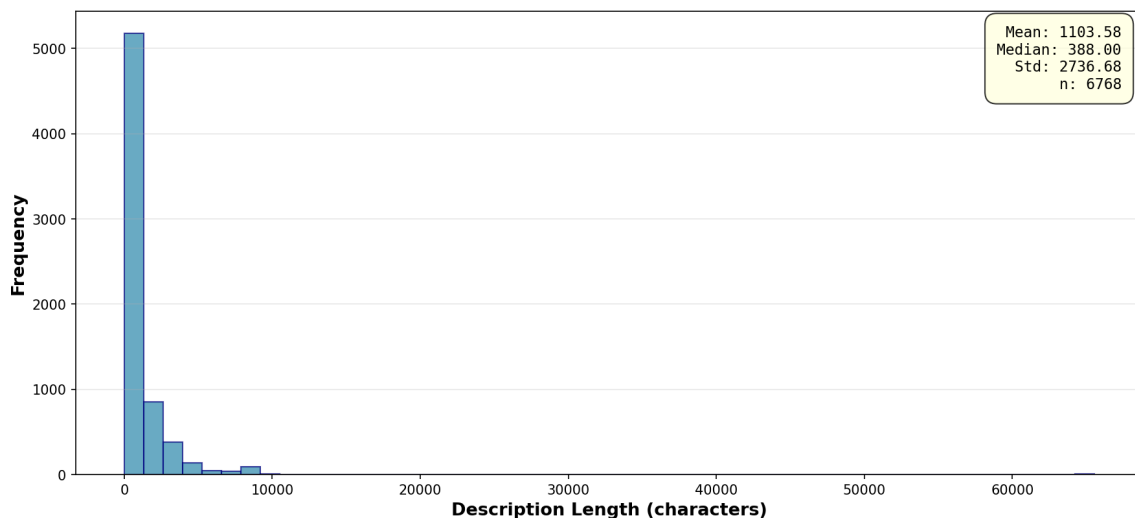
A figura apresenta a **distribuição do tamanho dos PRs** em linhas alteradas, considerando a soma de adições e remoções. Observa-se uma distribuição bastante assimétrica: a mediana é de apenas **8 linhas alteradas**, enquanto a média chega a **1.674,83 linhas**, indicando a presença de poucos PRs extremamente grandes que elevam a média. Essa diferença reforça que a mediana é mais representativa para descrever o PR típico da amostra.

Section 1: Temporal Metric: Analysis Time

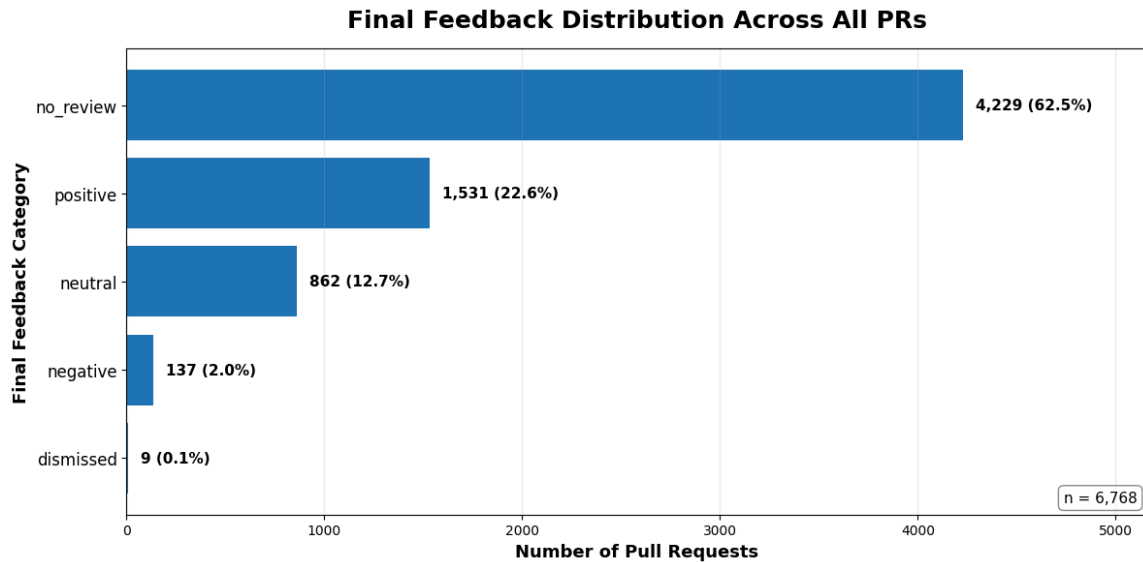


A figura mostra a **distribuição do tempo de análise dos PRs**. A mediana é de **69,77 horas**, enquanto a média é de **3.500,42 horas**, o que indica forte influência de PRs que permaneceram abertos por períodos muito longos. Essa visualização é importante para contextualizar as RQs relacionadas ao tempo de análise, pois demonstra que a duração dos PRs não segue uma distribuição uniforme.

Section 1: Description Metric: Length

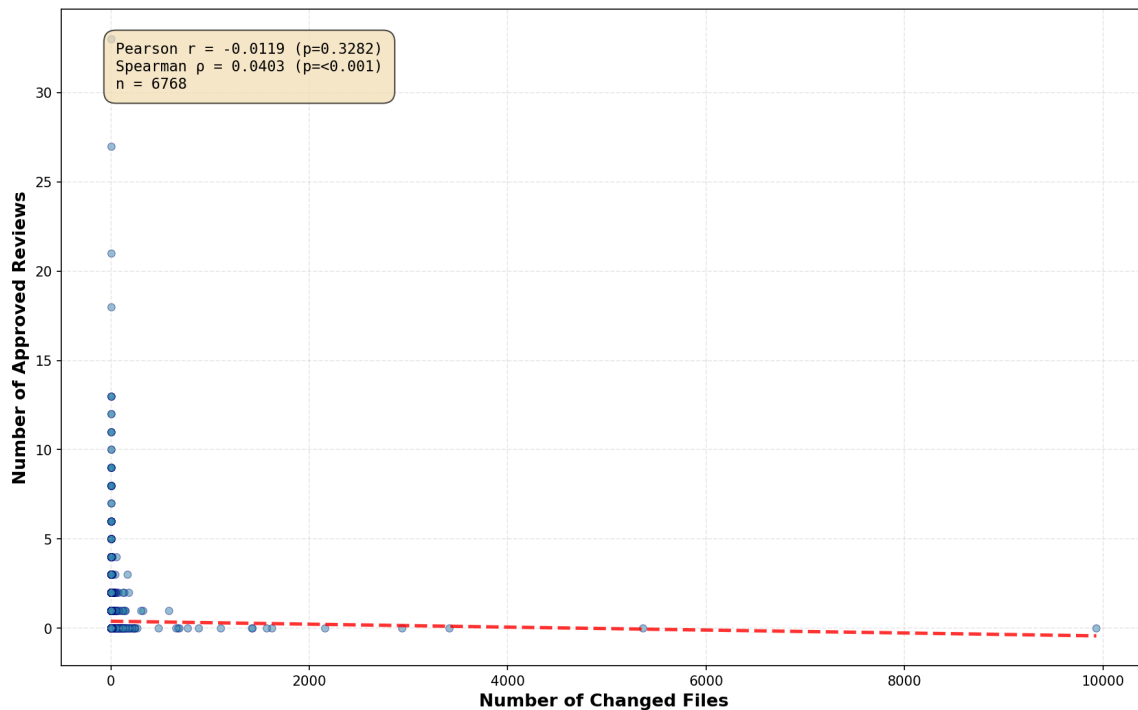


A figura apresenta a **distribuição do tamanho das descrições dos PRs**, medido em quantidade de caracteres. A mediana é de **388 caracteres**, enquanto a média é de **1.103,58 caracteres**, indicando que a maioria dos PRs possui descrições curtas, mas alguns casos com descrições muito longas deslocam a média para cima.



A figura mostra a distribuição do feedback final dos PRs. A maior parte dos registros aparece como **no_review**, representando **62,5%** da amostra. Em seguida, aparecem feedbacks positivos, com **22,6%**, neutros, com **12,7%**, negativos, com **2,0%**, e dismissed, com **0,1%**. Essa distribuição é relevante para interpretar as RQs 1 a 4, pois evidencia que os feedbacks negativos são pouco frequentes em comparação com os demais grupos.

Section 5: RQ5 - PR Size vs Review Count



A figura apresenta a relação entre o número de arquivos alterados e a quantidade de revisões aprovadas. Novamente, a correlação é muito fraca, indicando que PRs maiores não necessariamente recebem mais revisões. A concentração dos pontos próximos a poucos arquivos alterados e poucas revisões sugere que a maior parte dos PRs analisados é pequena e recebe baixa quantidade de interações formais de revisão.

5 - Discussão dos Resultados

Os resultados obtidos indicam que a **atividade de code review nos repositórios analisados apresenta forte assimetria nas principais métricas observadas**. A amostra contém 6.768 Pull Requests, com valores medianos bastante inferiores às médias em métricas como tamanho, tempo de análise e descrição. Por exemplo, o tamanho mediano dos PRs foi de apenas 8 linhas alteradas, enquanto a média foi de 1.674,83 linhas. De forma semelhante, o tempo mediano de análise foi de 69,77 horas, enquanto a média foi de 3.500,42 horas. **Esses resultados indicam que poucos PRs muito grandes ou muito demorados influenciam fortemente os valores médios.**

Em relação ao feedback final das revisões, observou-se uma predominância da categoria `no_review`, com 62,5% dos registros, seguida por feedbacks positivos, com 22,6%, neutros, com 12,7%, negativos, com 2,0%, e `dismissed`, com 0,1%. Essa distribuição mostra que feedbacks explicitamente negativos são pouco frequentes na

amostra, o que **limita a comparação direta entre categorias de feedback e reforça a necessidade de interpretar os resultados como tendências exploratórias.**

Considerando as questões RQ01 a RQ04, relacionadas ao feedback final, os resultados sugerem que as **características isoladas dos PRs não explicam de forma forte o resultado final da revisão.** A hipótese da RQ01, de que o tamanho do PR não influenciaria fortemente o feedback final, foi parcialmente confirmada, pois a visualização apresentada indica correlação muito fraca entre o número de arquivos alterados e a presença de revisão aprovada. **Isso sugere que PRs maiores não necessariamente recebem feedback mais positivo ou mais negativo apenas por causa do tamanho.**

A hipótese da RQ02 propunha que PRs com maior tempo de análise poderiam estar associados a mais dificuldades no processo de revisão e, portanto, a uma relação negativa com feedbacks positivos. Embora a distribuição do tempo de análise mostre grande variação e presença de outliers, esse comportamento deve ser interpretado com cuidado. PRs que ficam abertos por mais tempo podem representar alterações mais complexas, menor disponibilidade dos revisores, discussões prolongadas ou simplesmente atrasos no processo do projeto. **Assim, o tempo de análise parece ser uma métrica importante para caracterizar o processo, mas não deve ser interpretado isoladamente como indicador direto de pior feedback.**

Na RQ03, a hipótese inicial era de que o tamanho da descrição não influenciaria diretamente o feedback final. **A distribuição do tamanho das descrições reforça que a maioria dos PRs possui descrições relativamente curtas,** com mediana de 388 caracteres, apesar de existirem casos com descrições muito longas. Portanto, **uma descrição maior não necessariamente significa maior qualidade ou maior chance de aprovação.** A descrição pode ajudar o revisor a entender a mudança, mas seu efeito depende do conteúdo, da clareza e do contexto da alteração, não apenas da quantidade de caracteres.

Para a RQ04, esperava-se que mais interações pudessem indicar temas mais sensíveis ou controversos, aumentando a chance de feedbacks neutros ou negativos. Os valores medianos de interação indicam, porém, que o PR típico da amostra possui baixa interação: a mediana de comentários é 1 e a média é 2,82. Isso sugere que **grande parte dos PRs passa por um processo de revisão relativamente curto em termos de comentários, enquanto poucos casos concentram discussões mais longas.**

Nas questões RQ05 a RQ08, relacionadas ao número de revisões, os resultados também indicam associações fracas para as métricas analisadas individualmente. No caso da RQ05, a hipótese era de que PRs maiores poderiam demandar mais esforço de análise, mas também desestimular a participação de muitos revisores. **A visualização entre tamanho do PR e número de revisões mostra correlação muito fraca,** sugerindo que o tamanho, sozinho, não é suficiente para explicar a quantidade de revisões realizadas.

Na RQ06, a hipótese inicial indicava que PRs com maior tempo de análise poderiam atrair menos participação direta ou refletir processos de revisão mais lentos. **Os resultados descritivos mostram que há PRs com tempos de análise muito longos, mas isso não necessariamente significa maior número de revisões.** Esse comportamento pode estar relacionado a fatores externos ao PR, como disponibilidade dos mantenedores, prioridade da alteração ou práticas específicas de cada repositório.

Na RQ07, esperava-se que descrições mais completas pudessem atrair maior participação na revisão. **No entanto, como a métrica utilizada mede apenas quantidade de caracteres, ela não captura a qualidade real da descrição. Assim, os resultados devem ser interpretados com cautela: uma descrição longa pode ser mais informativa, mas também pode ser apenas um template automático, logs ou conteúdo pouco útil.** Para avaliar melhor essa hipótese, seria necessário analisar também a qualidade textual da descrição.

Por fim, na RQ08, a hipótese de que mais interações levariam a mais revisões é a mais intuitiva entre as propostas, pois comentários, participantes e revisões fazem parte do mesmo processo de discussão. Ainda assim, é importante separar interações informais, como comentários, de revisões formais registradas pelo GitHub. A relação esperada pode existir, mas deve ser avaliada considerando que diferentes projetos usam o mecanismo de revisão de maneiras distintas.

De modo geral, os resultados apontam que a atividade de *code review* é influenciada por múltiplos fatores e não pode ser explicada apenas por uma métrica isolada. As hipóteses relacionadas à baixa influência direta do tamanho e da descrição foram as mais compatíveis com os resultados observados. Já as hipóteses relacionadas a tempo e interações exigem uma análise mais detalhada, pois essas variáveis podem refletir tanto complexidade técnica quanto características sociais e organizacionais dos projetos.

6 - Conclusão

Este experimento analisou a atividade de *code review* em Pull Requests de repositórios populares do GitHub, buscando compreender como características dos PRs se relacionam com o feedback final das revisões e com o número de revisões realizadas. A análise foi conduzida sobre uma amostra de 6.768 PRs provenientes de 132 repositórios, utilizando métricas de tamanho, tempo de análise, descrição, interações, feedback final e número de revisões.

Os resultados mostram que a maioria das métricas analisadas possui distribuição bastante assimétrica. O PR típico da amostra tende a ser pequeno, com mediana de 8 linhas alteradas e apenas 1 arquivo modificado, mas há poucos PRs extremamente grandes que elevam a média. O mesmo comportamento aparece no tempo de análise e no tamanho das descrições, indicando que a mediana é mais adequada para representar o comportamento central da amostra.

Em relação ao feedback final, a maior parte dos PRs foi classificada como `no_review`, enquanto feedbacks negativos apareceram em proporção pequena. **Isso sugere que, na amostra analisada, rejeições ou avaliações explicitamente negativas não são o padrão mais comum. Além disso, as visualizações e correlações observadas indicam que métricas isoladas, como tamanho do PR, não explicam de forma forte o feedback final ou o número de revisões.**

Assim, conclui-se que as características quantitativas dos PRs ajudam a descrever o processo de *code review*, mas não são suficientes, sozinhas, para explicar completamente seus resultados. A atividade de revisão depende também de fatores contextuais, como normas do projeto, disponibilidade dos mantenedores, complexidade da mudança, urgência da correção e forma como cada comunidade utiliza os mecanismos de revisão do GitHub.

6.1 - Trabalhos futuros

Como trabalho futuro, uma primeira possibilidade é ampliar a coleta até atingir o universo planejado de 200 repositórios, reduzindo o impacto da amostra parcial e aumentando a representatividade dos resultados. Também seria útil separar os resultados por repositório ou por linguagem principal, permitindo verificar se projetos diferentes apresentam padrões distintos de revisão.

Outra melhoria seria refinar a análise do feedback final, separando melhor revisões humanas, comentários automáticos e eventos de bots. Embora o experimento tenha buscado tratar a revisão humana, a dinâmica real dos PRs pode envolver ferramentas automatizadas e diferentes tipos de eventos registrados pelo GitHub.

7 - Reprodutibilidade

A reprodutibilidade do experimento foi considerada desde a construção do pipeline. A coleta e a análise foram organizadas em scripts separados para descoberta de repositórios, extração de PRs, agregação dos dados, análise estatística e geração dos gráficos. A documentação da metodologia indica que os principais artefatos produzidos incluem o dataset agregado `aggregated_prs.csv`, os arquivos de estatísticas descritivas, a distribuição de feedbacks, as tabelas de correlação e os gráficos utilizados no relatório.

Os dados e scripts utilizados no experimento estão disponíveis no repositório informado no início do relatório:

<https://github.com/Pedro-nll/LabExperimentacaoDeSoftware/tree/enunciado3>

Para reproduzir a análise, deve-se executar o pipeline a partir da raiz do projeto, utilizando os comandos documentados para agregação dos dados, análise das correlações e geração dos gráficos. Os principais comandos são:

```
python enunciado3/scripts/aggregate_pr_data.py --input-dir enunciado3/output  
--output-csv enunciado3/output/aggregated_prs.csv
```

```
python enunciado3/scripts/correlation_analysis.py --input-csv  
enunciado3/output/aggregated_prs.csv --output-dir enunciado3/output/analysis
```

```
python enunciado3/scripts/analyze_rq_metrics.py --aggregated-csv  
enunciado3/output/aggregated_prs.csv --output-dir enunciado3/output/analysis
```

```
python enunciado3/scripts/generate_publication_plots.py --aggregated-csv  
enunciado3/output/aggregated_prs.csv --output-dir  
enunciado3/output/analysis/publication_plots
```

```
python enunciado3/scripts/report_pipeline.py
```

O comando **report_pipeline.py** é o ponto de entrada recomendado, pois centraliza a execução das principais etapas de agregação, análise estatística e geração dos resultados. A estratégia com checkpoints permite que execuções interrompidas sejam retomadas sem reprocessar todos os repositórios, preservando o andamento da coleta e facilitando novas execuções do experimento.